FPT Education

TRƯỜNG ĐẠI HỌC FPT

Module 7 - Forms and Validation

User Input • Validation • UX • Async Checks

Contents

- Build forms using Form and TextFormField
- Use FormState and GlobalKey to validate and save data
- Apply built-in and custom validation rules
- Manage focus and keyboard interactions
- Design user-friendly error messages
- Implement a complete signup form with validation
- This module leads into Module 8 – API Networking

Why Forms Matter

Why forms are critical in apps:

Common use cases:

- Login / Signup
- Profile update
- Feedback & contact forms
- Booking and checkout
- Search and filtering inputs

If forms are done poorly:

- Users submit invalid or incomplete data
- Users get frustrated and abandon the app
- Security and data quality issues appear

Key idea: Good apps = good forms + good validatio

Input Widgets Overview

Core input widgets in Flutter

- TextField – basic text input
- TextFormField – text input with validation in a form
- Checkbox, Switch, Radio – boolean choices
- DropdownButton – pick from a list of options
- Date & time pickers – select from dialogs

When to use which?

- Use TextField for simple, one-off inputs
- Use TextFormField inside a Form when you need validation

Form & FormState

What is a Form?

- A Form groups multiple input fields
- A GlobalKey<FormState> lets you:
  - Validate all fields at once
  - Save all field values
  - Reset the form

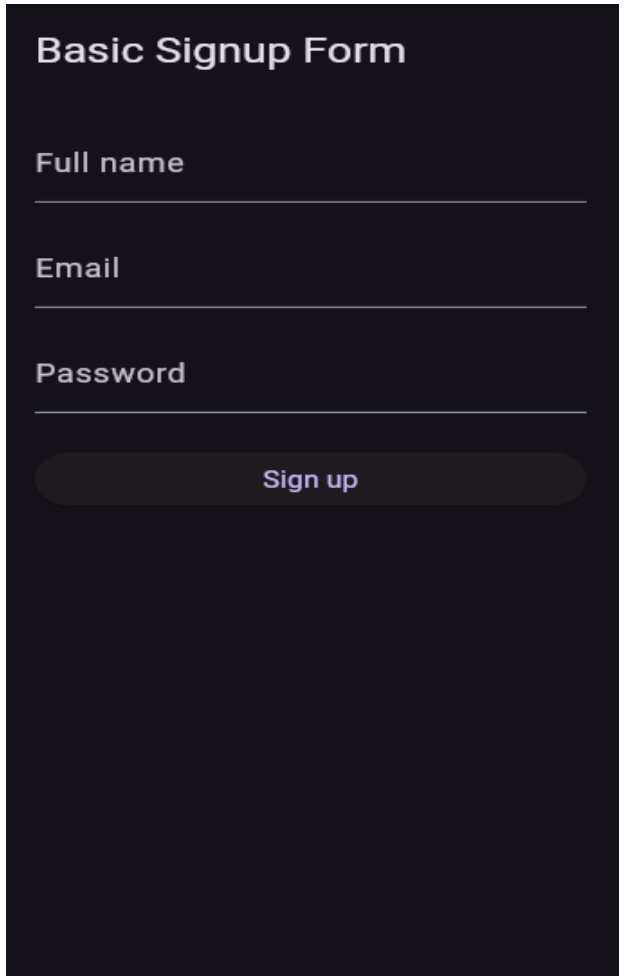
Core methods:

- formKey.currentState!.validate()
- formKey.currentState!.save()
- formKey.currentState!.reset()

```
final _formKey = GlobalKey<FormState>();
Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        decoration: const InputDecoration(labelText: 'Name'),
        validator: (value) {
          if (value == null || value.trim().isEmpty) {
            return 'Name is required';
          }
          return null;
        },
      ),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate()) {
            _formKey.currentState!.save();
          }
        },
        child: const Text('Submit'),
      ),
    ],
  ),
);
```

Demo 1- Basic Signup Form

- Goal: Create a simple signup form using Form + TextFormField.
- Requirements
  - Fields: Name – Email – Password
  - All fields required
  - Email must contain @ and .
  - Password ≥ 6 characters
- Submit Behavior
  - validate() → show inline errors
  - If valid → show SnackBar: “Signup successful”



Basic Signup Form - Structure

- **Minimal signup screen structure**
  - Scaffold
  - AppBar
  - Form with _formKey
  - TextFormFields for name, email, password
  - A submit button

```
Scaffold(
  appBar: AppBar(title: const Text('Basic Signup Form')),
  body: Padding(
    padding: const EdgeInsets.all(16),
    child: Form(
      key: _formKey,
      child: ListView(
        children: [
          TextFormField(
            decoration: const InputDecoration(labelText: 'Full name'),
          ),
          TextFormField(
            decoration: const InputDecoration(labelText: 'Email'),
          ),
          TextFormField(
            decoration: const InputDecoration(labelText: 'Password'),
            obscureText: true,
          ),
          const SizedBox(height: 24),
          ElevatedButton(
            onPressed: _submit,
            child: const Text('Sign up'),
          ),
        ],
      ),
    ),
  ),
);
```

Basic Signup Form - Built-in Validators

- **The validator function**
  - Signature: String? Function(String? value)
  - If valid → return null
  - If invalid → return error message (String)
- **Common checks:**
  - Required field (not empty)
  - Email format (contains @ and .)
  - Minimum length

```
TextFormField(
  decoration: const InputDecoration(labelText: 'Email'),
  validator: (value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return 'Email is required';
    if (!text.contains('@') || !text.contains('.')) {
      return 'Enter a valid email';
    }
    return null;
  },
);
```

Basic Signup Form - Submit Flow & Button State

- **Typical submit flow:**
  - User taps Submit
  - Call validate()
  - If valid → call save() and submit data
  - If invalid → show inline errors
  - Display error when form invalid
  - Display SnackBar when valid
  - "FormState standard" Submit logic
- **(Simple submit flow → validate → save → SnackBar**

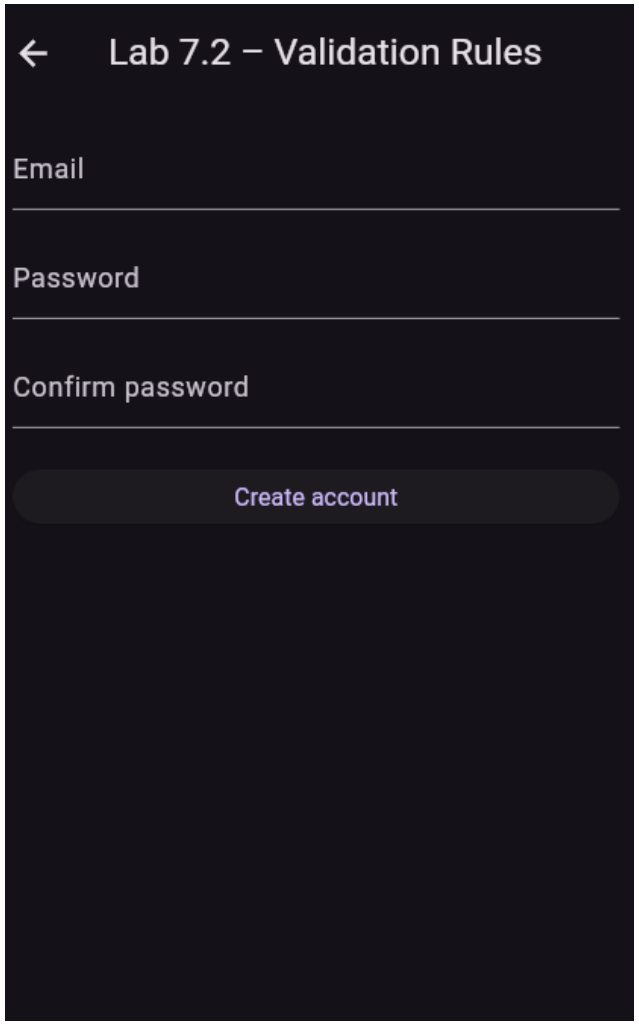
```
void _submit() {
  final isValid = _formKey.currentState!.validate();
  if (!isValid) return;

  _formKey.currentState!.save();

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Form is valid!')),
  );
}
```

Demo 2 - Validation Rules

- Goal: Add strong password + confirm password logic.
- Requirements:
  - Password ≥ 8 chars & includes number
  - Confirm password must match
  - Use helper validator functions
  - Enable AutovalidateMode.onUserInteraction



Demo 2 - Custom Validation Rules

- **Use custom validators for:**
  - Business rules (username rules, banned words)
  - Password strength (length, numbers, symbols)
  - More advanced email checks (pattern / regex)

```
String? validatePassword(String? value) {
  final text = value ?? '';
  if (text.isEmpty) return 'Password is required';
  if (text.length < 8) return 'At least 8 characters';
  if (!text.contains(RegExp(r'[0-9]'))) {
    return 'Add at least one number';
  }
  return null;
}

String? validateConfirmPassword(String? value, String password) {
  if (value == null || value.isEmpty) return 'Please confirm password';
  if (value != password) return 'Passwords do not match';
  return null;
}
```

Demo 2 - Confirm Password Logic

- Why confirm password?
  - Prevent typos
  - Improve user confidence
- Pattern:
  - Keep _password in state
  - Compare confirm password with _password
  - Show error if they differ

```
TextFormField(
  decoration: const InputDecoration(labelText: 'Confirm password'),
  obscureText: true,
  onChanged: (value) => _confirmPassword = value,
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Please confirm password';
    }
    if (value != _password) {
      return 'Passwords do not match';
    }
    return null;
  },
);
```

Demo 2 - AutovalidateMode

- **Autovalidate modes:**
  - AutovalidateMode.disabled
  - AutovalidateMode.always
  - AutovalidateMode.onUserInteraction (recommended)
- **UX trade-offs:**
  - Too early → annoying (red errors everywhere from the start)
  - Too late → confusing (errors appear only after submit)

```
Form(
  key: _formKey,
  autovalidateMode: AutovalidateMode.onUserInteraction,
  child: Column(
    children: [ /* fields */ ],
  ),
);
```

Error Messages & UX

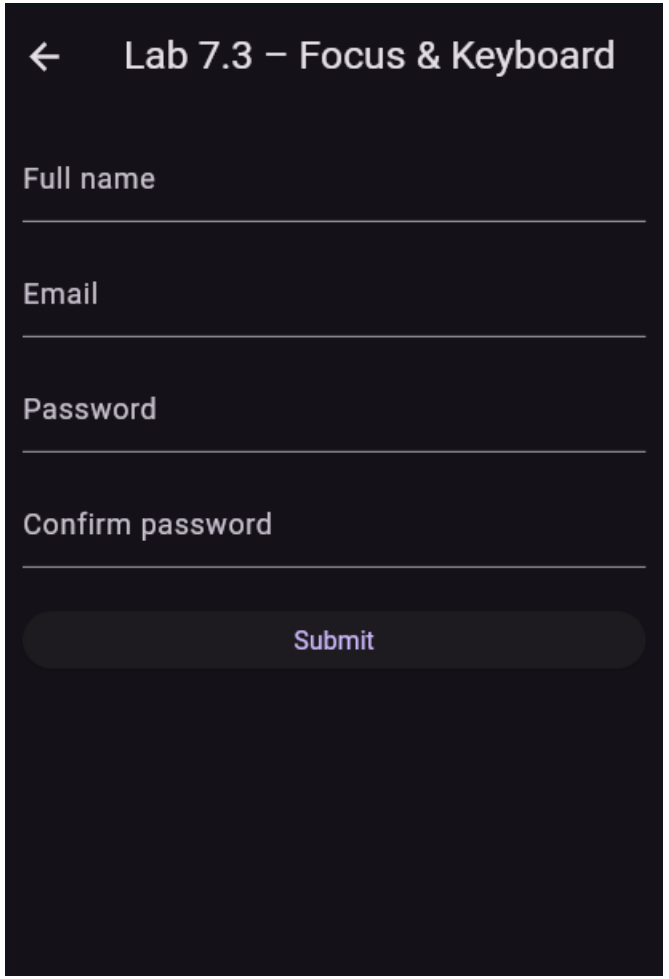
- Good error messages:
  - Short and clear
  - Specific to the field
  - Polite and helpful
- Examples:
  - “Enter a valid email”
  - “Password must be at least 8 characters”
  - “ERROR_400_INVALID_FIELD”

Best practices:

- Show error under the field
- Avoid blaming language
- Highlight only relevant fields

Demo 3 - Focus & Keyboard UX

- **Goal:**
  - Improve input experience on mobile.
- **Requirements:**
  - Create FocusNode for each field
  - Use Next/Done with textInputAction
  - Move focus using onFieldSubmitted
  - Dismiss keyboard using GestureDetector
  - Wrap form with ListView to prevent overflow



Demo 3 - Managing Focus & Keyboard Content

- Why manage focus?
  - Mobile keyboards can hide fields
  - Users expect Next/Done to work smoothly
- Tools:
  - FocusNode
  - textInputAction
  - onFieldSubmitted
  - FocusScope.of(context).requestFocus(nextFocusNode)

```
final _nameFocus = FocusNode();
final _emailFocus = FocusNode();

TextFormField(
  focusNode: _nameFocus,
  textInputAction: TextInputAction.next,
  onFieldSubmitted: (_) {
    FocusScope.of(context).requestFocus(_emailFocus);
  },
);

TextFormField(
  focusNode: _emailFocus,
  textInputAction: TextInputAction.done,
);
```

Demo 3 - Dismissing the Keyboard

- Common UX requirement:
  - Tap outside the form → keyboard hides
  - Pattern: Wrap with GestureDetector
- Also:
  - Wrap form in ListView or SingleChildScrollView to avoid overflow.

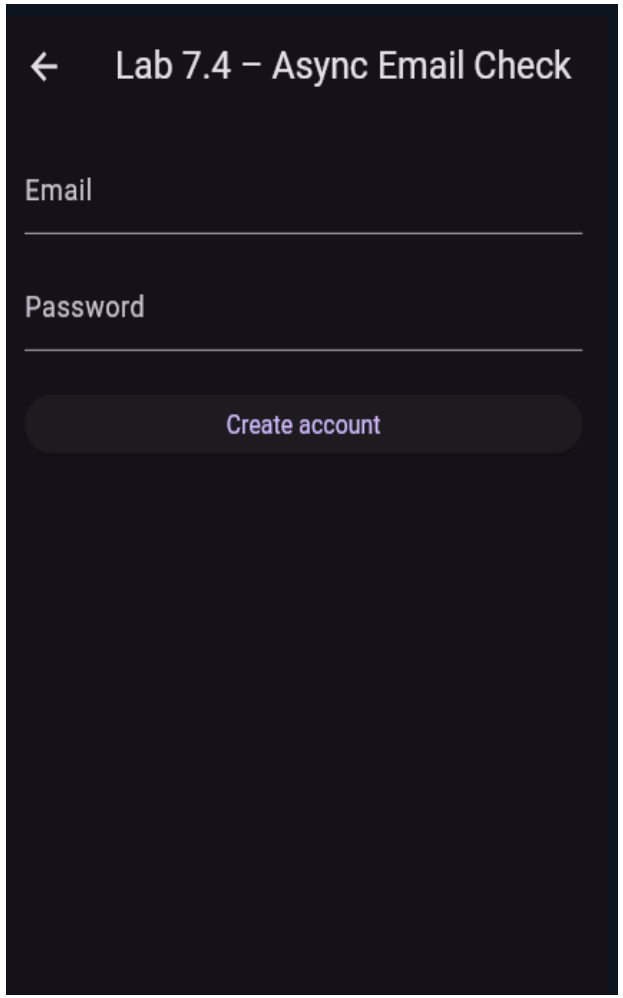
```
return GestureDetector(
  onTap: () => FocusScope.of(context).unfocus(),
  child: Scaffold(
    appBar: AppBar(title: const Text('Focus & Keyboard')),
    body: /* Form here */,
  ),
);
```

Form Layout Patterns

- **Common layouts:**
  - Single-page scrollable form
  - Grouped sections (e.g. Personal Info / Account Info)
  - Multi-step wizard (optional, conceptual)
- **Avoid:**
  - Too many fields on screen without grouping
  - Keyboard covering the submit button
  - Fixed height containers that don’t scroll

Demo 4 - Async Email Check (Optional)

- **Goal:**
  - Simulate server-side validation before account creation.
- **Requirements**
  - Validate form locally
  - Show loading state during async check
  - Delay 2s: Future.delayed()
  - If email starts with “taken” → show error: “Email already taken”
  - Disable Submit button during check



Async Email Check (Optional)

Why async validation?

- Check if email/username already exists
- Validate codes or coupons
- Confirm availability (e.g. booking slot)

- **Pattern (simplified):**

```
bool _isCheckingEmail = false;
Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;
  setState(() => _isCheckingEmail = true);
  await Future.delayed(const Duration(seconds: 2));
  final emailTaken = _email.toLowerCase().startsWith('taken');
  setState(() => _isCheckingEmail = false);
  if (emailTaken) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('This email is already taken')),
    );
    return;
  }
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('Account created for $_email')),
  );
}
```

From Form to Backend (Suggested Content)

What happens after validation?

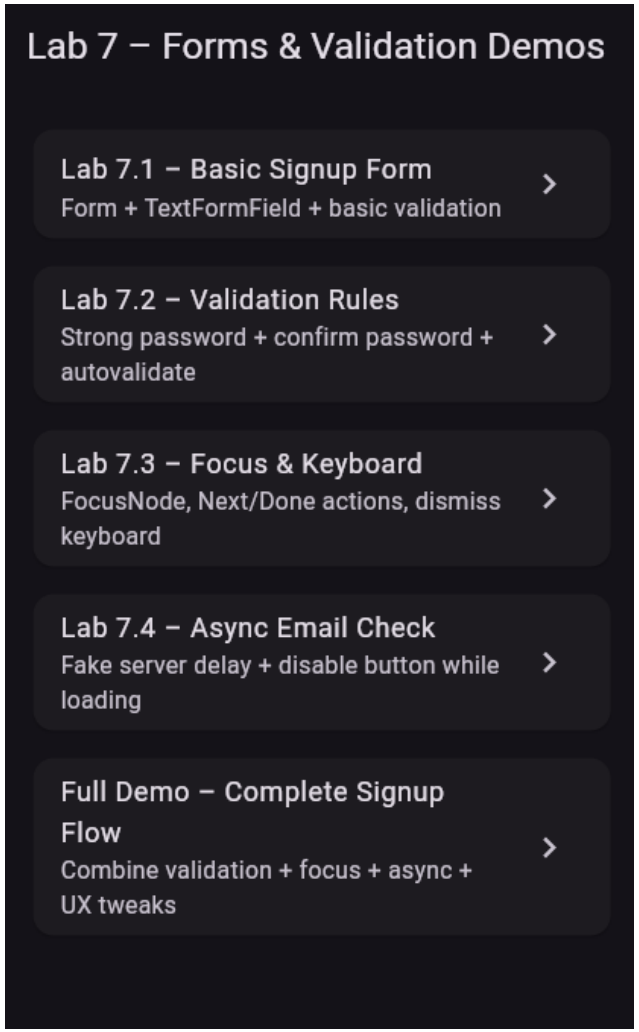
- User fills the form
- Local validation checks (Module 7)
- Send data via API (Module 8)
- Server performs deeper checks
- Server returns success/error
- UI updates accordingly

UI Form → Validation → POST Request → Server → Response → UI Update

Full Signup Demo (Combined)

Features combined:

- Name, email, password, confirm password
- Built-in + custom validation
- Autovalidate on user interaction
- Focus management (Next/Done)
- Show/hide password
- Async email check with loading button



Lab Forms & Validation

Lab 7 – Forms & Validation

- Lab 7.1 – Basic registration form
- Lab 7.2 – Validation rules & password strength
- Lab 7.3 – Focus & keyboard management
- Lab 7.4 – Optional async email check

- **You will:**
  - Implement the signup form step by step
  - Run it on DartPad and on emulator/device
  - Capture screenshots and submit your code

Summary

In this module, you learned how to:

- Build forms using Form and TextFormField
- Use FormState to validate, save, and reset data
- Implement required fields and custom validation rules
- Manage focus and keyboard interactions on mobile
- Show clear error messages and success feedback
- Combine everything into a complete signup form

References

- Mastering Flutter 2025 – Ch.4, 14
- docs.flutter.dev
- **https://dartpad.dev**